

Algorithms and Data Structures I.

Sample Second Test

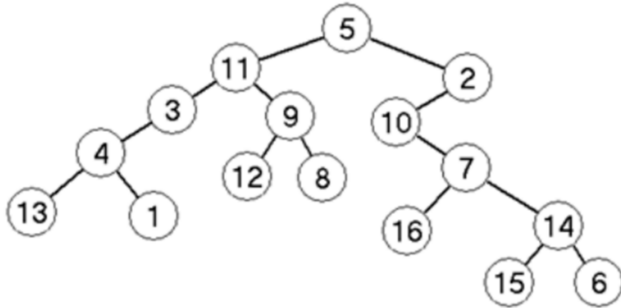
Note: The sample test was created with examples from the practice class material.
On the real test, if the task contains multiple *Options*, it will be chosen randomly

Task 1. (4 points):

a) Provide the order of elements according to the 4 types of traversal strategies!

Note: This task will ask for all 3 simple traversals and the level-order traversal:

Preorder, Inorder, Postorder, Level-order



b) During the traversal of a non-complete binary tree, the following sequences were obtained;
reconstruct the shape of the tree.

POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3

INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72

Task 2. (8 points):

a) Construct a **Binary Search Tree** with the insertion of the following elements:

$\langle 19, 17, 30, 10, 21, 12, 42, 25, 33, 34 \rangle$

Demonstrate the final tree of the data structure after the final insertion!

d) Delete the keys 12 and then 30 from the tree, maintaining the *Binary Search Tree (BST)* structure property. Demonstrate the tree after each of the deletions separately.

Task 3. (12 points):

Sort the array with **HeapSort**. Note where the comparisons happen with the arrows, and write *swap* when the elements have to be swapped. After demonstrating this on the tree, redraw the tree to demonstrate the result of the iteration.

$\langle 4, 1, 3, 2, 18, 9, 10, 14, 12, 7 \rangle$

a) Demonstrate the BUILDMAXHEAP() algorithm.

b) Demonstrate the first 2 iterations of the SORTING() algorithm.

Task 4. (12 points):

Note: This task will be one of the 3 open addressing hash table probe sequences:

Linear, Quadratic, Double hashing

Given an **open-addressed hash table** of size 8 using a quadratic probe sequence.

Use the hash function:

$$h(k, n) = (k + \frac{n+n^2}{2}) \bmod 8.$$

Demonstrate how the table changes with every operation!

Operation	k	$h_1(k)$	Probe sequence	Result	0	1	2	3	4	5	6	7
Insert	13											
Insert	20											
Insert	31											
Insert	87											
Insert	12											
Delete	31											
Search	12											
Search	15											
Insert	4											
Delete	10											
Insert	35											
Insert	10											

Task 5. (14 points):

- a) Note: This task will be the implementation of a **learned** operation related to *Binary trees*.
Present your algorithm in structogram format!

Given a **Binary tree** (t), provide an algorithm that returns the number of nodes having a single child in the first **k** level.

- b) Note: This task will be the implementation of a **not learned** operation related to *Binary trees*.
Present your algorithm in structogram format!